Lesson 1.3 — The Mindset Shift

Lesson 1.3 — The Mindset Shift

> **The interviewer can usually tell within 60 seconds whether a candidate uses AI as a *tool* or as a *crutch*.** This lesson is the difference.

By the end of this lesson:

- You'll know the single phrase that signals you've internalized the senior shift.
- You'll be able to recognize — in your own behavior and in others' — the four "tells" of a crutch user.
- You'll have a concrete habit list you can adopt this week to make your shift visible.
- You'll have a story-shaped answer ("Tell me about a time you didn't trust the AI…") that lands every time.

---

1. The shift in one sentence

> **Junior with AI:** "What does the AI think I should do?"
>
> **Senior with AI:** "What do *I* think we should do? And how can the AI help me move faster on that?"

That's the shift. Internal locus of judgment. The AI is your typist, your sounding board, your researcher, your reviewer — but it is **not** your decider.

If you remember nothing else from this entire course, remember that. **The AI is not the decider.**

---

2. The four tells of a crutch user

These are the patterns interviewers watch for. They're easy to spot. **Audit yourself for them.**

2.1. The "let me ask the AI" reflex

The crutch user, when asked a question they don't know the answer to, immediately reaches for the AI **before thinking**. They don't form an initial hypothesis. They don't articulate what they don't know. They just paste-and-pray.

The senior user thinks first. *"What's the shape of this problem? What do I already know? What's the smallest question I'd need to answer to make progress?"* Only then do they prompt — and the prompt is **specific** because the thinking happened first.

> *Interview equivalent:* You're asked a hard system-design question. The crutch user immediately says *"Could I look this up?"* The senior says *"Let me think out loud for a minute,"* sketches a first-pass approach on the whiteboard, then later says *"In a real working context I'd validate this design with an LLM and pressure-test it against alternatives."*

Same outcome. Different signal.

2.2. The "the AI said so" defense

The crutch user, when asked *why* their code does X, says: *"That's how the AI suggested it."*

Three problems:

1. It's not an explanation; it's an attribution.
2. It signals you didn't read it carefully.
3. It moves accountability away from you, which is the opposite of what a senior does.

The senior user *can always explain why*. If they can't, they don't ship. **Inability to explain a piece of code in your codebase is a personal red flag.**

> *Interview equivalent:* You're walking through your portfolio project. The interviewer asks why you used `useMemo` on a particular value. *"That's what the AI generated"* loses the room. *"Because the parent re-renders on every keystroke in the search box, and without the memo the chart's data prop changes identity, which would cascade into the chart re-rendering"* — that wins the room.

2.3. The diff-flood acceptance

The crutch user accepts giant diffs without reading them. They watch the AI rewrite 14 files, see "tests pass," and move on. The senior user **forces small diffs**. They reject "rewrite this whole module" suggestions in favor of "extract this one function" steps they can review.

> *In any agentic interview, the senior moves the AI in small steps. The crutch user runs the AI on a long task and reviews the result en masse.*

2.4. The novelty assumption

The crutch user assumes anything the AI suggests is current best practice. The senior user knows the AI's training data lags reality by months — sometimes years — and that "best practice" is contextual anyway.

> *Concrete example:* The AI may still suggest `useState` patterns from 2022 when your app would benefit from React 19 server actions. It may suggest `pages/` Next.js routes when your codebase is in `app/`. It may suggest a deprecated library because that library was popular when the model was trained.

Senior tell: "I don't accept dependency suggestions from the AI without checking the package's recent activity and that we don't already have a similar package."

---

3. The phrase that signals the shift

When an interviewer asks you about a code change you made, work this phrase in:

> *"My instinct was X. I asked the AI for a second opinion / a sanity check / an alternative, and it suggested Y. I went with [X / Y / a hybrid] because [reason rooted in your project's context]."*

That phrase, in three lines, conveys:

1. You had your own instinct first. **Internal locus.**
2. You used the AI as a *peer*, not as an oracle.
3. You made the final call yourself, citing **your** context.

Compare to the crutch version:

"I asked the AI and it told me to do X."

The two sentences describe the same outcome. The first one gets you the offer.

---

4. The "did you read what the AI wrote?" question

This will be asked. Sometimes directly:

"Did you actually read every line of code in your portfolio project?"

Sometimes obliquely:

"Walk me through this function." → They pick the most non-obvious one.

The right answer is *"Yes,"* and the proof is that you can walk through any line on demand.

The defense:

- **Spend an hour with your portfolio code before any interview.** Read every component. Star anything you don't fully recall. Refresh.
- **If you find code you can't explain, rewrite it.** Even if it's working. The interview goal is to be able to explain *every* line. AI-generated code is fair game; AI-generated code you can't explain is not.

This is the tax of using AI fluently. Pay it.

---

5. The four habits that signal the shift

Adopt these in your daily work *now* and they will leak into your interview answers naturally.

5.1. Form a hypothesis before you prompt

Before typing a single character to the AI, write down (in your head or in a comment) what *you* think the answer is. Even a wrong hypothesis is useful — it gives you a baseline to compare the AI's answer against.

5.2. Always read the diff

Even tiny diffs. *Especially* tiny diffs — the small ones are where sneaky behavioral changes hide. Make this a reflex, not a chore. Senior engineers can spot a one-character bug in a 200-line diff because they've read enough diffs.

5.3. Run the code before trusting it

The AI is confident even when wrong. The compiler isn't. The test runner isn't. **Run the code.** Run the tests. The number of senior engineers who've shipped a confident-looking AI output without running it is *not* zero, and they'll all tell you the same horror story when you ask.

5.4. Keep a log of AI mistakes

For one week, write down every time the AI was wrong. The format: *what you asked, what it returned, what was wrong, what the right answer was*. By Friday you'll have a personal failure-mode catalog. **It will be your best interview prep artifact.**

---

6. Story arsenal: "Tell me about a time you didn't trust the AI…"

This question (or a variant) shows up in roughly 60% of interviews that mention AI. **Have a story ready.** Use the STAR format.

> **Situation:** "I was working on the scoring engine for a fantasy-sports app. Each team has picks across five rounds, and the scoring needed to handle both completed and pending matches."
>
> **Task:** "I needed a function that gracefully handled the *pending* case — not as a zero, but as a separate state — because the UI shows three different colors for correct, wrong, and pending."
>
> **Action:** "I asked the AI to write the scoring function. It returned a clean implementation that returned 0 for unplayed matches. That was wrong for my use case. The bug wouldn't have surfaced in tests of completed tournaments — only during a live tournament. I rejected the diff, prompted again with the explicit constraint *'unplayed matches must return null, not zero, because the UI distinguishes pending from wrong'*, and accepted the second pass."

> **Result:** "The function works correctly throughout the tournament lifecycle. The lesson I keep coming back to: the AI optimizes for the spec it was given, and any constraint I forget to mention is a constraint it can't satisfy."

Memorize a story like this from your own work. **Make sure the story has a moment where you exercised judgment the AI couldn't.** That moment is the interview's gold.

---

7. The "what would you do if AI got 10× better tomorrow?" question

A wildcard question, but increasingly asked. Have a structured answer.

> *"I'd shift more time to design and judgment. The bottom of the engineering stack — typing, implementation — would get even cheaper. The top — choosing what to build, designing for our specific constraints, owning the trade-offs — would not. Those are the layers I'd invest in. Concretely: more user research, more architectural-trade-off documentation, more time talking to other teams, less time looking at code I personally typed. The AI raises the floor of what's possible. It doesn't raise the ceiling of what's required from a senior."*

---

8. Three exercises for this week

8.1. The hypothesis log

For the next 5 working days, write down your hypothesis *before* prompting the AI. At the end of the week, look at the log. How often were you right? How often was the AI? What patterns of disagreement do you see?

8.2. The diff drill

Pick the next 10 AI-generated diffs that land in your code. **Force yourself to articulate what each one changes, line by line, before accepting it.** It will feel slow. After 10 diffs, it will be habit.

8.3. The story file

Open `interview-stories.md`. Write down 5 STAR-format stories from your real work in the last 6 months. At least 2 of them should feature a moment where you did *not* trust the AI and made the call yourself.

---

9. Common pitfalls in this conversation

9.1. The performative caution

> *"Oh, I'm very careful with AI. I read everything. I run all the tests. I never just accept things."*

If asked specifically what you check for, the crutch user goes vague. **Have specifics.** Otherwise it sounds rehearsed in the bad way.

9.2. The performative skepticism

> *"AI is interesting but I generally write everything myself."*

Translation in the interviewer's head: *"This person is slow."* Don't.

9.3. The performative depth

> *"I deeply understand transformer architecture and use that to inform my prompting."*

Unless they asked, this is irrelevant trivia. **Don't lecture about ML internals.** Talk about engineering outcomes.

9.4. Apologizing for AI use

> *"Yeah, I… used AI for some of this, sorry."*

Don't apologize. AI use is the modern default. **Frame it as professional craft, not a confession.**

---

10. CHECK YOURSELF

- ☐ Can you state the one-sentence shift (section 1) from memory?
- ☐ Can you list the four “tells” of a crutch user and explain each in your own words?
- ☐ Can you give the *senior version* of the phrase from section 3 about your own work?
- ☐ Have you done at least one of the three exercises in section 8?
- ☐ Do you have a STAR story from your real work where you didn’t trust the AI?

If yes to all five, you’ve completed the foundations chapter. **Onward.**

---

11. Where you are now

After this chapter you have:

- A vocabulary for AI-assisted development with five tool generations and three workflows.
- A five-layer model that makes seniority visible.
- A list of new skills that command a premium.
- A mindset shift you can feel and project in conversation.
- An exercise list that takes a week to complete and changes how you work.

This is your platform. Chapter 2 builds the most important answer you’ll ever give in an interview: the multi-pillar response to *“Why do we still need human developers?”*

Open **../chapter-02-the-big-question/README.md** when you’re ready.